Kubernetes Operators with Go

Operator Framework — From Concept to Implementation

01

What are Operators?

Encoding operational knowledge into software

What are Operators?

"An Operator is an application-specific controller that extends the Kubernetes API to manage complex application instances."

Encapsulates Knowledge

Codifies SRE expertise into software controllers

Full Automation

Deploy, scale, backup, upgrade — all automated

Native K8s Extension

Uses CRDs + Controllers
First-class K8s citizen

Operators in the Wild

Prometheus Operator	Manages Prometheus, AlertManager, ServiceMonitors — zero manual config
PostgreSQL Operator	Automated backups, failover, connection pooling, rolling upgrades
Cert-Manager	Automated TLS certificate provisioning and renewal via Let's Encrypt
Istio Operator	Full Istio service mesh lifecycle management including upgrades

02

Controller Pattern & CRDs

The building blocks behind every Operator

How It Works

CRD	Custom Resource Definition — Extends K8s API with your own resource types (e.g., MyApp, PostgresCluster)
Controller	Watches for changes on resources and runs the Reconcile loop to converge desired ↔ current state
Operator	CRD + Controller together — a custom controller that manages the full lifecycle of a custom resource

03

Operator Framework & SDK

The toolkit that makes building Operators easy

Operator Framework

Open-source toolkit by Red Hat / CNCF to build, test, and package Operators

Operator SDK

Scaffolding, code gen, and testing tools. Supports Go, Ansible, or Helm operators

OLM

Operator Lifecycle Manager. Handles install, upgrade, RBAC, and dependencies

OperatorHub

Public registry of community and certified operators. Ready to deploy

Getting Started with Operator SDK

```
# Initialize a new operator project
$ operator-sdk init \
    --domain gfnogueira.com \
    --repo github.com/nogueira/myapp-operator

# Create a new API (CRD + Controller)
$ operator-sdk create api \
    --group app --version v1alpha1 \
    --kind MyApp --resource --controller
```

1. Init Project

Go module, Makefile, Dockerfile, RBAC

2. Create API

CRD types file + controller scaffold

3. Define Spec

Edit myapp_types.go with your fields

4. Implement Logic

Write Reconcile() in the controller

Project structure: api/v1alpha1/myapp_types.go • controllers/myapp_controller.go • config/ (crd, rbac, manager) • Dockerfile • main.go

Defining Your Custom Resource

api/v1alpha1/myapp_types.go

```go
// MyAppSpec defines the desired state
type MyAppSpec struct {
    // Number of replicas for the app
    Replicas int32  `json:"replicas"`
    // Container image to deploy
    Image    string `json:"image"`
    // Port the app listens on
    Port     int32  `json:"port,omitempty"`
}

// MyAppStatus defines the observed state
type MyAppStatus struct {
    Conditions    []metav1.Condition `json:"conditions"`
```

What the user writes:

```yaml
apiVersion:
app.gfnogueira.com/v1alpha1
kind: MyApp
metadata:
  name: my-web-app
spec:
  replicas: 3
  image: nginx:1.25
  port: 8080
```

Spec = what user wants
Status = what actually exists

K8s stores both in etcd

The Reconcile Loop

The heart of every Operator — triggered on every change

```go
func (r *MyAppReconciler) Reconcile(
    ctx context.Context, req ctrl.Request,
) (ctrl.Result, error) {
    // 1. Fetch the CR instance
    myapp := &appv1.MyApp{}
    if err := r.Get(ctx, req.NamespacedName, myapp); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }
    // 2. Check if Deployment exists, create if not
    deploy := &appsv1.Deployment{}
    err := r.Get(ctx, req.NamespacedName, deploy)
    if errors.IsNotFound(err) {
        deploy = r.buildDeployment(myapp)
        return ctrl.Result{}, r.Create(ctx, deploy)
    }
    // 3. Ensure replicas match desired state
    if *deploy.Spec.Replicas != myapp.Spec.Replicas {
        deploy.Spec.Replicas = &myapp.Spec.Replicas
        return ctrl.Result{}, r.Update(ctx, deploy)
    }
    return ctrl.Result{}, nil  // All good!
}
```

Live Demo

MyApp Operator in action

Best Practices & When to Use

When to Use / Best Practices

- → Stateful apps (DBs, caches, queues)
- → Complex lifecycle management
- → Domain-specific automation
- → Keep Reconcile() idempotent
- → Use Status subresource
- → Add Finalizers for cleanup
- → Write tests with envtest

When NOT to Use / Avoid

- ✗ Simple stateless apps (use Deployment)
- ✗ One-off tasks (use Jobs/CronJobs)
- ✗ Side effects in Reconcile
- ✗ Hardcoded namespaces / configs
- ✗ Ignoring error handling
- ✗ Skipping RBAC least-privilege
- ✗ Over-engineering simple problems

Recap & Resources

- Operator SDK docs → sdk.operatorframework.io
- Kubebuilder book → book.kubebuilder.io
- controller-runtime → sigs.k8s.io/controller-runtime
- OperatorHub → operatorhub.io

Operators = CRD + Controller + Domain Knowledge → Kubernetes-native automation

Thank you!

Guilherme Nogueira

References;

- https://sdk.operatorframework.io
- https://book.kubebuilder.io
- https://pkg.go.dev/sigs.k8s.io/controller-runtime
- https://operatorhub.io